

Generalisation in theory and practice

Christos Dimitrakakis

September 29, 2026

Outline

Quality metrics

- Supervised machine learning problems

Metrics for model quality

- Classification

- Regression

Generalisation

Estimating quality

- Methodology

Learning and generalisation

- Introduction

- Bias and variance

- Generalisation

PAC Learning

- The realisable setting

Quality metrics

Supervised machine learning problems

Metrics for model quality

Classification

Regression

Generalisation

Estimating quality

Methodology

Learning and generalisation

Introduction

Bias and variance

Generalisation

PAC Learning

The realisable setting

Classification

The classifier as a decision rule

A decision rule $\pi(a|x)$ generates a **decision** $a \in [m]$. It is the conditional probability of a given x .

Classification

The classifier as a decision rule

A decision rule $\pi(a|x)$ generates a **decision** $a \in [m]$. It is the conditional probability of a given x .

Deterministic predictions given a model $P(y|x)$

Here, we pick the most likely class:

$$\pi(a|x_t) = \mathbb{I} \left\{ a = \arg \max_y P(y|x_t) \right\}$$

Classification

The classifier as a decision rule

A decision rule $\pi(a|x)$ generates a **decision** $a \in [m]$. It is the conditional probability of a given x .

Deterministic predictions given a model $P(y|x)$

Here, we pick the most likely class:

$$\pi(a|x_t) = \mathbb{I} \left\{ a = \arg \max_y P(y|x_t) \right\}$$

Randomised predictions given a model $P(y|x)$

Here, we randomly select a class according to our model:

$$\pi(a|x_t) = P(y_t = a|x_t)$$

Accuracy as a classification metric

Accuracy as a classification metric

The accuracy of a single decision

$$U(a_t, y_t) = \mathbb{I}\{a_t = y_t\} = \begin{cases} 1, & \text{if } a_t = y_t \\ 0, & \text{otherwise} \end{cases}$$

Accuracy as a classification metric

The accuracy of a single decision

$$U(a_t, y_t) = \mathbb{I}\{a_t = y_t\} = \begin{cases} 1, & \text{if } a_t = y_t \\ 0, & \text{otherwise} \end{cases}$$

The accuracy on a dataset

Let $D = \{(x_t, y_t) : t \in [T]\}$ be a dataset. We can measure the accuracy:

$$U(\pi, D) \triangleq \frac{1}{T} \sum_{t=1}^T \pi(y_t | x_t)$$

The expected accuracy of a decision rule

Accuracy as a classification metric

The accuracy of a single decision

$$U(a_t, y_t) = \mathbb{I}\{a_t = y_t\} = \begin{cases} 1, & \text{if } a_t = y_t \\ 0, & \text{otherwise} \end{cases}$$

The accuracy on a dataset

Let $D = \{(x_t, y_t) : t \in [T]\}$ be a dataset. We can measure the accuracy:

$$U(\pi, D) \triangleq \frac{1}{T} \sum_{t=1}^T \pi(y_t | x_t)$$

The expected accuracy of a decision rule

If $(x, y) \sim P$, the accuracy U of a stochastic decision rule π under the distribution P is the probability it predicts correctly

$$U(\pi, P) \triangleq \int_{\mathcal{X}} dP(x) \sum_{y=1}^m P(y|x) \pi(y|x)$$

Beyond classification: Generalised decision rules

Consider a spam application, where the e-mail client can decide between different action for emails. Different actions being best for each type of e-mail. The quality of each action can be captured through a utility function.

Utility of the spam decision problem

What utility function would you use for the spam detection problem?

Utility	Pass	Flag	Trash
Normal			
Spam			
Virus			

The utility function $U : \mathcal{Y} \times \mathcal{A} \rightarrow \mathbb{R}$

The utility function $U(y, a)$ is a real-valued function so that, for a label y , we prefer taking action a to a' iff $U(y, a) > U(y, a')$.

The optimal decision

- ▶ A **model** $P(y|x)$ of class probabilities
- ▶ A **utility** $U(y, a)$ for each class and action combination

Expected utility

We can calculate the expected utility of any decision

$$\mathbb{E}[U|a, x] = \sum_y P(y|x, a)U(y, a) = \sum_y P(y|x)U(y, a)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

The optimal decision

- ▶ A **model** $P(y|x)$ of class probabilities
- ▶ A **utility** $U(y, a)$ for each class and action combination

Expected utility

We can calculate the expected utility of any decision

$$\mathbb{E}[U|a, x] = \sum_y P(y|x, a)U(y, a) = \sum_y P(y|x)U(y, a)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

The optimal decision

For any observation x , and P , we take the action maximising expected utility:

$$a^* = \arg \max_a \mathbb{E}_P[U|a, x]$$

This defines a function $\mathcal{X} \rightarrow \mathcal{A}$, which is the Bayes-optimal decision rule.

The optimal decision rule

- ▶ A **model** $P(y|x)$ of class probabilities
- ▶ A **utility** $U(y, a)$ for each class and action combination
- ▶ A **decision rule** $\pi(a|x)$ assigning probability to action a for every possible input x

Expected utility over a dataset.

We obtain the expected utility of the **decision rule** by marginalising over all actions

$$U(\pi, D) \triangleq \mathbb{E}[U|\pi, D] \stackrel{D=(x_t, y_t)_{t=1}^T}{=} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[U|\pi, y_t, x_t] = \frac{1}{T} \sum_{t=1}^T \sum_{a \in \mathcal{A}} U(y_t, a) \pi(a|x_t)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

Expected utility over P

We can marginalise over possible datasets D

$$U(\pi, P) \triangleq \mathbb{E}_P^\pi[U] = \int_{\mathcal{D}} dP(D) U(\pi, D) \stackrel{\text{i.i.d.}}{=} \int_{\mathcal{X}} dP(x) \sum_y P(y|x) \sum_a \pi(a|x) U(y, a)$$

Generalisation in theory and practice

Quality metrics

Supervised machine learning problems

The optimal decision rule

The optimal decision rule

- A **model** $P(y|x)$ of class probabilities
- A **utility** $U(y, a)$ for each class and action combination
- A **decision rule** $\pi(a|x)$ assigning probability to action a for every possible input x

Expected utility over a dataset.

We obtain the expected utility of the **decision rule** by marginalising over all actions

$$U(\pi, D) \triangleq \mathbb{E}[U|\pi, D] \stackrel{D \sim (x_t, y_t)_{t=1}^T}{=} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[U|\pi, y_t, x_t] = \frac{1}{T} \sum_{t=1}^T \sum_{a \in \mathcal{A}} U(y_t, a) \pi(a|x_t)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

Expected utility over P

We can marginalise over possible datasets D

$$U(\pi, P) \triangleq \mathbb{E}_P[U] = \int_D dP(D) U(\pi, D) \stackrel{D \sim (x_t, y_t)_{t=1}^T}{=} \int_{\mathcal{X}} dP(x) \sum_y P(y|x) \sum_a \pi(a|x) U(y, a)$$

Explanation. The utility on a dataset is fixed once we know the dataset. The expectation in $U(\pi, D)$ is taken over the randomness of π itself.

We define $U(\pi, P)$ in terms of the randomness of both P and π . First, we fix the decision rule π . Then, we sample a dataset randomly $D \sim P$. This makes $U(\pi, D)$ random.

To make things more concrete, the formulas expand for the case the utility is a sum of the utilities over individual examples, with $D = (x_t, y_t)_{t=1}^T$

However, **irrespective** of whether or not the utility is decomposed into a sum of per-example utilities, the definition

$$U(\pi, P) \triangleq \int_D dP(D) U(\pi, D)$$

says that $U(\pi, P)$ is the expected utility if we randomly draw some dataset D from P .

Generalisation in theory and practice

Quality metrics

Supervised machine learning problems

The optimal decision rule

The optimal decision rule

- A **model** $P(y|x)$ of class probabilities
- A **utility** $U(y, a)$ for each class and action combination
- A **decision rule** $\pi(a|x)$ assigning probability to action a for every possible input x

Expected utility over a dataset.

We obtain the expected utility of the **decision rule** by marginalising over all actions

$$U(\pi, D) \triangleq \mathbb{E}[U(\pi, D)] \stackrel{D \sim \{x_1, y_1\}^T}{=} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[U(\pi, y_t, x_t)] = \frac{1}{T} \sum_{t=1}^T \sum_{a \in A} U(y_t, a) \pi(a|x_t)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

Expected utility over P

We can marginalise over possible datasets D

$$U(\pi, P) \triangleq \mathbb{E}_P[U] = \int_{\mathcal{D}} dP(D) U(\pi, D) \stackrel{(a)}{=} \int_{\mathcal{X}} dP(x) \sum_y P(y|x) \sum_a \pi(a|x) U(y, a)$$

Notes on the derivation: To keep the slide compact, the full derivation of how we go from datasets to a single random example was omitted. Here it is, for completeness.

$$\begin{aligned} \int_{\mathcal{D}} dP(D) U(\pi, D) &= \int_{\mathcal{D}} d \left[\prod_t P(x_t, y_t) \right] U(\pi, D) \\ &= \int_{\mathcal{D}} d \left[\prod_t P(x_t, y_t) \right] \frac{1}{T} \sum_t \mathbb{E}[U \mid \pi, x_t, y_t] \\ &\stackrel{(a)}{=} \int_{\mathcal{D}} d \frac{1}{T} \sum_t P(x, y) \mathbb{E}[U \mid \pi, x, y] = \int_{\mathcal{X} \times \mathcal{Y}} dP(x, y) \mathbb{E}[U \mid \pi, x, y]. \end{aligned}$$

For the last step, it suffices to note that $P(x, y) = P(y|x)P(x)$, so that:

$$\int_{\mathcal{X} \times \mathcal{Y}} P(x, y) \mathbb{E}[U \mid \pi, x, y] = \int_{\mathcal{X}} dP(x) \sum_{y \in \mathcal{Y}} P(y|x) \sum_{a \in A} \pi(a|x) U(a, y)$$

Generalisation in theory and practice

Quality metrics

Supervised machine learning problems

The optimal decision rule

The optimal decision rule

- A **model** $P(y|x)$ of class probabilities
- A **utility** $U(y, a)$ for each class and action combination
- A **decision rule** $\pi(a|x)$ assigning probability to action a for every possible input x

Expected utility over a dataset

We obtain the expected utility of the **decision rule** by marginalising over all actions

$$U(\tau, D) \triangleq \mathbb{E}[U(\tau, D)] \stackrel{\text{do } (x_i, y_i)_{i=1}^T}{=} \frac{1}{T} \sum_{i=1}^T \mathbb{E}[U(\tau, y_i, x_i)] = \frac{1}{T} \sum_{i=1}^T \sum_{a \in \mathcal{A}} U(y_i, a) \pi(a|x_i)$$

Here the first equality follows from the definition of conditional expectation and $P(y|x, a) = P(y|x)$ as the label does not depend on our actions.

Expected utility over P

We can marginalise over possible datasets D

$$U(\tau, P) \triangleq \mathbb{E}_P[U] = \int_D dP(D) U(\tau, D) \stackrel{11.6}{=} \int_{\mathcal{X}} dP(x) \sum_y P(y|x) \sum_a \pi(a|x) U(y, a)$$

More notes on step (a). The proof idea is the same as the following example:

$f(x, z) = g(x) + g(z)$, and $P(x, z) = P(x)P(z)$.

$$\begin{aligned} \sum_{x, z} P(x, z) f(x, z) &= \sum_{x, z} P(x) P(z) [g(x) + g(z)] \\ &= \sum_x P(x) \sum_z P(z) [g(x) + g(z)] \\ &= \sum_x P(x) \sum_z P(z) g(x) + \sum_x P(x) \sum_z P(z) g(z) \\ &= \sum_x P(x) g(x) \sum_z P(z) + \sum_x P(x) \sum_z P(z) g(z) \\ &= \mathbb{E}_P[g] \times 1 + \mathbb{E}_P[g] \times 1 = 2 \mathbb{E}_P[g]. \end{aligned}$$

Essentially, each term in the sum gets linked to one random element, and the others sum to one.

Quality metrics

Supervised machine learning problems

Metrics for model quality

Classification

Regression

Generalisation

Estimating quality

Methodology

Learning and generalisation

Introduction

Bias and variance

Generalisation

PAC Learning

The realisable setting

In fact, we may want to simply find a good model, and do not have a specific decision problem to solve. How should we select a model?

The basic accuracy metric we had for classification decision rules was just to count whether or not we classified something correctly. This made sense, as our decision was a distinct action or label. We used a utility function to modify this in a way that reflects our preferences. However, we sometimes want to instead measure the **accuracy of our model**. What is the best way to do that?

For classification, it makes sense to look at the label probability. If we are not very confident about our prediction, this should be taken into account. Let $P(y|x)$ be our model's probability for label y , given features x . Then we can use two simple metrics: precision and log-loss.

Label probability is also useful for selecting a regression model. However, in that case, a notion of distance from the target is also useful. The two are actually equivalent in some cases.

Precision

The average probability of the actual class:

$$\sum_{t=1}^T P(y_t|x_t) / T$$

- ▶ If we always assign probability 1 to the correct label, this score is 1.
- ▶ If we always assign probability $1/m$ to all labels, the score is $1/m$.
- ▶ Exercise: if our model P is correct, and we always decide the maximum-probability class, show that the accuracy (as previously defined) is the same with precision.

Negative Log-Loss

Here we assign look at the **logarithm** of the probability. This really penalises bad guesses.

- ▶ $\sum_{t=1}^T \ln P(y_t|x_t) / T$
- ▶ If we always assign probability 1 to the correct label, this score is 0.
- ▶ If we assign probability 0 to even a single true label, the score is $-\infty$.

```
from sklearn.metrics import log_loss
```

in scikitlearn implements log-loss (**not** negative)

Regression

The regressor as a deterministic decision rule

A decision rule π generates a **decision** $a \in \mathbb{R}^m$.

- ▶ For **deterministic** rules $\pi(x)$ is the prediction for x .
- ▶ We need to define the quality of our predictions, either as a utility $U(y_t, a_t)$ (to maximise) or a loss function $\ell(y_t, a_t)$ (to minimise).

Mean-Squared Error Loss on a Dataset

This is the squared difference in predicted versus actual values:

$$\frac{1}{T} \sum_{t=1}^T [y_t - \pi(x_t)]^2$$

Expected MSE

If $(x, y) \sim P$, the expected MSE of a deterministic rule $\pi : \mathcal{X} \rightarrow \mathbb{R}$ is:

$$\mathbb{E}(y - \pi)^2 = \int_{\mathcal{X}} \mathbb{E}[(y - \pi(x))^2 \mid x] dP(x) = \int_{\mathcal{X}} \int_{\mathcal{Y}} dP(x, y) [y - \pi(x)]^2.$$

Probabilistic regression

The regressor as a stochastic decision rule

A decision rule π generates a **decision** $a \in \mathbb{R}^m$.

- ▶ For **stochastic** rules $\pi(a|x)$ defines a density over predictions.
- ▶ In this case it is natural to define $\pi(y_t | x_t)$ as our metric.

Likelihood on a Dataset

The likelihood is simply the probability of the labels

$$\prod_{t=1}^T \pi(y_t | x_t)$$

We will later see a link between this metric, mean-square error and estimation.

Quality metrics

Supervised machine learning problems

Metrics for model quality

Classification

Regression

Generalisation

Estimating quality

Methodology

Learning and generalisation

Introduction

Bias and variance

Generalisation

PAC Learning

The realisable setting

Training and overfitting

Training data

- ▶ $D = ((x_t, y_t) : t = 1, \dots, T)$.
- ▶ $x_t \in \mathcal{X}, y_t \in \mathcal{Y}$.

Training and overfitting

Training data

- ▶ $D = ((x_t, y_t) : t = 1, \dots, T)$.
- ▶ $x_t \in \mathcal{X}, y_t \in \mathcal{Y}$.

Assumption: The data is generated i.i.d.

- ▶ $(x_t, y_t) \sim P$ for all t (identical)
- ▶ $D \sim P^T$ (independent)

Training and overfitting

Training data

- ▶ $D = ((x_t, y_t) : t = 1, \dots, T)$.
- ▶ $x_t \in \mathcal{X}, y_t \in \mathcal{Y}$.

Assumption: The data is generated i.i.d.

- ▶ $(x_t, y_t) \sim P$ for all t (identical)
- ▶ $D \sim P^T$ (independent)

The optimal decision rule for P

$$\max_{\pi} U(\pi, P) = \max_{\pi} \int_{\mathcal{X} \times \mathcal{Y}} dP(x, y) \sum_a \pi(a|x) U(a, y).$$

Here we measure performance for the data **distribution P** .

Training and overfitting

Training data

- ▶ $D = ((x_t, y_t) : t = 1, \dots, T)$.
- ▶ $x_t \in \mathcal{X}, y_t \in \mathcal{Y}$.

Assumption: The data is generated i.i.d.

- ▶ $(x_t, y_t) \sim P$ for all t (identical)
- ▶ $D \sim P^T$ (independent)

The optimal decision rule for P

$$\max_{\pi} U(\pi, P) = \max_{\pi} \int_{\mathcal{X} \times \mathcal{Y}} dP(x, y) \sum_a \pi(a|x) U(a, y).$$

Here we measure performance for the data **distribution** P .

The optimal decision rule for D

$$\max_{\pi} U(\pi, D) = \max_{\pi} \sum_{(x, y) \in D} \sum_a \pi(a|x) U(a, y).$$

Performance is measured over the **data** D

Generalisation

The fundamental problem

- ▶ We want to maximise $U(\pi, P)$, the **actual** performance.
- ▶ We can only measure $U(\pi, D)$, over **some data**.
- ▶ We have a **learning algorithm** $\lambda : \mathcal{D} \rightarrow \Pi$
- ▶ If $\pi = \lambda(D)$, then we instead measure $U(\lambda(D), D)$.
- ▶ However $\mathbb{E}_P[U(\lambda(D), D)] \geq U(\pi, P) \forall \pi$ (**biased** estimator)

Generalisation

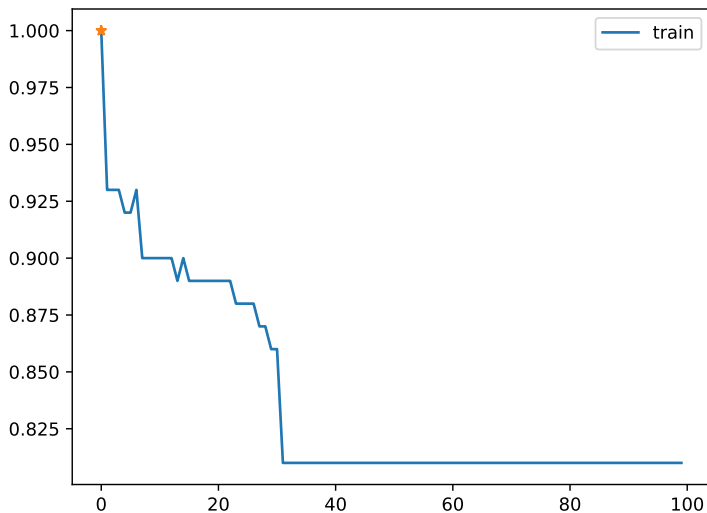
The fundamental problem

- ▶ We want to maximise $U(\pi, P)$, the **actual** performance.
- ▶ We can only measure $U(\pi, D)$, over **some data**.
- ▶ We have a **learning algorithm** $\lambda : \mathcal{D} \rightarrow \Pi$
- ▶ If $\pi = \lambda(D)$, then we instead measure $U(\lambda(D), D)$.
- ▶ However $\mathbb{E}_P[U(\lambda(D), D)] \geq U(\pi, P) \forall \pi$ (**biased** estimator)

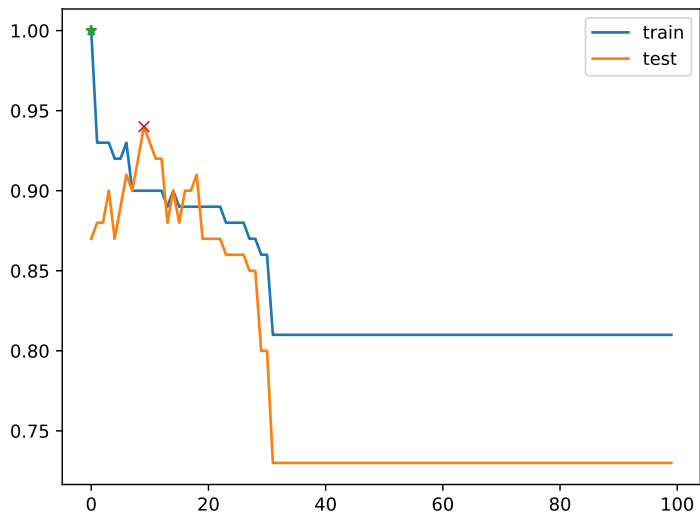
Training and testing

- ▶ Split D in $D_{\text{train}}, D_{\text{test}}$
- ▶ Obtain $\pi = \lambda(D_{\text{train}})$
- ▶ Calculate $U(\pi, D_{\text{test}})$
- ▶ $\mathbb{E}_P[U(\pi, D_{\text{test}})] = U(\pi, P)$ for any fixed π (unbiased estimator)
- ▶ $\mathbb{E}_P[U(\lambda(D_{\text{train}}), D_{\text{train}})] \geq U(\pi, P)$ (biased estimator)

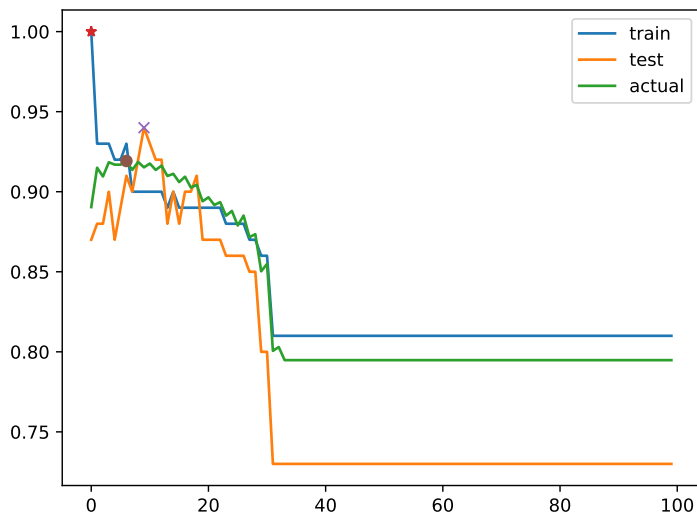
kNN Classifier Accuracy on a single dataset



kNN Classifier Accuracy on a single dataset

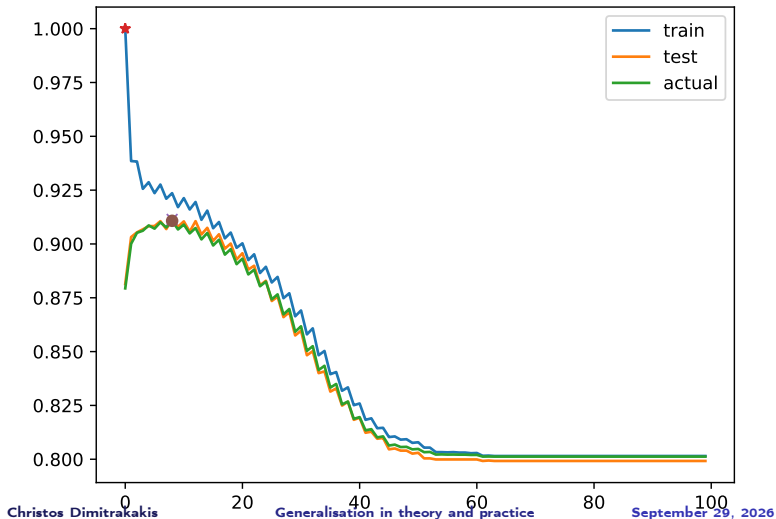


kNN Classifier Accuracy on a single dataset



Expected kNN Classifier Accuracy

Expectation approximated over 100 datasets D sampled from P .



Quality metrics

Supervised machine learning problems

Metrics for model quality

Classification

Regression

Generalisation

Estimating quality

Methodology

Learning and generalisation

Introduction

Bias and variance

Generalisation

PAC Learning

The realisable setting

The Train/Validation/Test methodology

Main idea

Use each piece of data once to make decisions and measure

Training set

Use to decide low-level model parameters

Validation set

Use to decide between:

- ▶ different hyperparameters (e.g. K in nearest neighbours)
- ▶ model (e.g. neural networks versus kNN)

Test set

Use to measure the final quality of a model

Cross-validation (XV)

Idea

- ▶ Use XV to select hyperparameters instead of a single train/valid test.

Methodology

- ▶ Split training set D in k different subsets
- ▶ At iteration i
- ▶ Use the i -th subset for validation
- ▶ Use all the remaining $k - 1$ subsets for training
- ▶ Average results on validation sets

Cross-validation example: A mean estimate

Live coding in Python:

- ▶ Get a mean estimate
- ▶ Perform cross-validation

Hand-crafted

```
def xv_shuffler(x, estimator, scoring, n_folds):  
    rng = np.random.default_rng  
    # shuffle data  
    T = len(x)  
    indices = np.arange(T)  
    shuffle(indices)  
    fold_size = np.ceil(T / n_folds)  
    fold_start = np.zeros(n_folds)  
    fold_end = np.zeros(n_folds)  
    # create folds  
    for k in range(n_folds):  
        fold_start = k * fold_size  
        fold_end = np.min((k+1) * fold_size, T)  
    # for each fold:
```

1. Run estimator on $x_{1:n}$ folds

Bootstrapping

- ▶ Express uncertainty by resampling the data.
- ▶ Repeat your calculations for each resample

Bootstrapping

- ▶ Express uncertainty by resampling the data.
- ▶ Repeat your calculations for each resample

BootstrapSample(D)

```
input Data  $D = (z_1, \dots, z_T)$ , of size  $T$   
for  $t \in \{1, \dots, T\}$  do  
    Select  $i$  uniformly in  $[T]$   
    Add the  $i$ -th point to  $D_b$   
end for  
return  $D_b$ 
```


Bootstrapping

- ▶ Express uncertainty by resampling the data.
- ▶ Repeat your calculations for each resample

BootstrapSample(D)

```

input Data  $D = (z_1, \dots, z_T)$ , of size  $T$ 
for  $t \in \{1, \dots, T\}$  do
    Select  $i$  uniformly in  $[T]$ 
    Add the  $i$ -th point to  $D_b$ 
end for
return  $D_b$ 
  
```

BootstrapEstimate(D, λ, N)

```

input Data  $D \in \mathcal{D}$ , algorithm  $\lambda : \mathcal{D} \rightarrow \Theta$ ,  $N > 0$  number of samples
for  $n \in \{1, \dots, N\}$  do
     $\theta_n = \lambda(\text{BootstrapSample}(D))$ 
end for
return  $\{\theta_n : n \in [N]\}$ 
  
```

The wrong way to do XV for subset selection

1. Screen the features: find a subset of “good” features that show fairly strong (univariate) correlation with the class labels.
2. Using just this subset of features, build a multivariate classifier.
3. Use cross-validation to estimate the unknown tuning parameters and to estimate the prediction error of the final model.

Is this a correct application of cross-validation?

Consider a scenario with $N = 50$ samples in two equal-sized classes, and $p = 5000$ quantitative features that are independent of the class labels (e.g. normally distributed). The true error rate of any classifier in that set is 50%. Can you still find some “good” classifiers?

The right way to do XV for feature selection

1. Divide the samples into K cross-validation folds (groups) at random.
2. For each fold $k = 1, 2, \dots, K$
3. Find a subset of “good” features that show fairly strong (univariate) correlation with the class labels, using all of the samples except those in fold k .
4. Using just this subset of features, build a multivariate classifier, using all of the samples except those in fold k .
5. Use the classifier to predict the class labels for the samples in fold k .

Generalisation in theory and practice

└ Estimating quality

└ Methodology

└ The right way to do XV for feature selection

The right way to do XV for feature selection

1. Divide the samples into K cross-validation folds (groups) at random.
2. For each fold $k = 1, 2, \dots, K$.
3. Find a subset of "good" features that show fairly strong (univariate) correlation with the class labels, using all of the samples except those in fold k .
4. Using just this subset of features, build a multivariate classifier, using all of the samples except those in fold k .
5. Use the classifier to predict the class labels for the samples in fold k .

Level notes

- **BSc:** Be careful not to leak information from another fold.
- **MSc:** **Why** does the wrong version give a near-zero apparent error despite a true error of 50%? Connect it to the estimation-error bound above: what is $|\hat{\Pi}|$ effectively doing once you've screened 5000 features first?

Lab

- ▶ scikitlearn KNN
- ▶ scikitlearn performance measures
- ▶ train/test/validate plot with increasing k
- ▶ XV plot with increasing k
- ▶ Scaling and other preprocessing
- ▶ Effect of outliers on preprocessing
- ▶ Optional: Bootstrap performance evaluation

Quality metrics

Supervised machine learning problems

Metrics for model quality

Classification

Regression

Generalisation

Estimating quality

Methodology

Learning and generalisation

Introduction

Bias and variance

Generalisation

PAC Learning

The realisable setting

Learning and generalisation

How well can decision rule perform?

Estimation theory view

- ▶ Bias: The **expected difference** between the estimated value and the unknown parameter
- ▶ Variance: Just the **variance** of that estimator, as a random variable.
- ▶ Related to generalisation error.

Learning theory view

- ▶ Approximation error: How well a class of rules can approximate the optimal one.
- ▶ Estimation error: How easy it is to choose the best rule in the class.

Generalisation in theory and practice

Learning and generalisation

Introduction

Learning and generalisation

Learning and generalisation

How well can decision rule perform?

Estimation theory view

- Bias: The **expected difference** between the estimated value and the unknown parameter
- Variance: Just the **variance** of that estimator, as a random variable.
- Related to generalisation error.

Learning theory view

- Approximation error: How well a class of rules can approximate the optimal one.
- Estimation error: How easy it is to choose the best rule in the class.

Which one do we use? This course mainly follows the **learning theory view**: it talks directly about $U(\pi, P)$, the quantity we actually care about, rather than about estimating some parameter θ . It bounds the generalisation error directly. The **estimation theory view** is covered later for completeness. It's a standard statistics framing, where we care about how well we can estimate a parameter, rather than how well we perform. It can be used to indirectly bound the generalisation error.

Unbiased estimators

Definition (Estimator)

An estimator is a function $f : \mathcal{D} \rightarrow \Theta$, where Θ is a set of parameters. For any given dataset $D \in \mathcal{D}$, it returns a single estimate $\hat{\theta} = f(D)$.

Definition (Unbiased estimator)

An estimator is **unbiased** if, for any θ and corresponding distribution $P(D|\theta)$:

$$\mathbb{E}[f \mid \theta] = \sum_D f(D)P(D|\theta) = \theta.$$

Example (Sample mean estimator)

Consider $D = (x_1, \dots, x_T)$ with $x_t \sim P$ being i.i.d samples with $\mathbb{E}[x_t] = \theta$. The sample mean estimator $f(D) = \sum_t x_t / T$ is unbiased, as :

$$\mathbb{E}[f] = \mathbb{E} \left[\sum_{t=1}^T x_t / T \right] = \frac{1}{T} \sum_{t=1}^T \mathbb{E}[x_t] = \frac{1}{T} \sum_{t=1}^T \theta = \theta.$$

Example of a biased and unbiased estimator: Training error

- ▶ $U(\pi, D)$ is the measured accuracy of a classifier π on data D

(4)

- ▶ i.e. the expected training accuracy is higher than that of any classifier.

Example of a biased and unbiased estimator: Training error

- ▶ $U(\pi, D)$ is the **measured** accuracy of a **classifier** π on **data** D
- ▶ $U(\pi, P) = \mathbb{E}_{D \sim P}[U(\pi, D)]$ is the **actual** accuracy on distribution P .

(4)

- ▶ i.e. the expected training accuracy is higher than that of any classifier.

Example of a biased and unbiased estimator: Training error

- ▶ $U(\pi, D)$ is the **measured** accuracy of a **classifier** π on **data** D
- ▶ $U(\pi, P) = \mathbb{E}_{D \sim P}[U(\pi, D)]$ is the **actual** accuracy on distribution P .
- ▶ So $U(\pi, D)$ is an unbiased estimate of $U(\pi, P)$.

(4)

- ▶ i.e. the expected training accuracy is higher than that of any classifier.

Example of a biased and unbiased estimator: Training error

- ▶ $U(\pi, D)$ is the **measured** accuracy of a **classifier** π on **data** D
- ▶ $U(\pi, P) = \mathbb{E}_{D \sim P}[U(\pi, D)]$ is the **actual** accuracy on distribution P .
- ▶ So $U(\pi, D)$ is an unbiased estimate of $U(\pi, P)$.
- ▶ $\lambda(D) = \arg \max_{\pi} U(\pi, D)$ is a **learning algorithm**, picking the best classifier for a dataset D .

(4)

- ▶ i.e. the expected training accuracy is higher than that of any classifier.

Example of a biased and unbiased estimator: Training error

- ▶ $U(\pi, D)$ is the **measured** accuracy of a **classifier** π on **data** D
- ▶ $U(\pi, P) = \mathbb{E}_{D \sim P}[U(\pi, D)]$ is the **actual** accuracy on distribution P .
- ▶ So $U(\pi, D)$ is an unbiased estimate of $U(\pi, P)$.
- ▶ $\lambda(D) = \arg \max_{\pi} U(\pi, D)$ is a **learning algorithm**, picking the best classifier for a dataset D .
- ▶ Then $U(\lambda(D), D)$ is biased, as its expectation obeys:

$$\mathbb{E}_{D \sim P}[U(\lambda(D), D)] = \int_D dP(D) U(\lambda(D), D) \quad (1)$$

(4)

- ▶ i.e. the expected training accuracy is higher than that of any classifier.

Example of a biased and unbiased estimator: Training error

- ▶ $U(\pi, D)$ is the **measured** accuracy of a **classifier** π on **data** D
- ▶ $U(\pi, P) = \mathbb{E}_{D \sim P}[U(\pi, D)]$ is the **actual** accuracy on distribution P .
- ▶ So $U(\pi, D)$ is an unbiased estimate of $U(\pi, P)$.
- ▶ $\lambda(D) = \arg \max_{\pi} U(\pi, D)$ is a **learning algorithm**, picking the best classifier for a dataset D .
- ▶ Then $U(\lambda(D), D)$ is biased, as its expectation obeys:

$$\mathbb{E}_{D \sim P}[U(\lambda(D), D)] = \int_D dP(D) U(\lambda(D), D) \quad (1)$$

$$= \int_D dP(D) \max_{\pi} U(\pi, D) \quad (2)$$

(4)

- ▶ i.e. the expected training accuracy is higher than that of any classifier.

Example of a biased and unbiased estimator: Training error

- ▶ $U(\pi, D)$ is the **measured** accuracy of a **classifier** π on **data** D
- ▶ $U(\pi, P) = \mathbb{E}_{D \sim P}[U(\pi, D)]$ is the **actual** accuracy on distribution P .
- ▶ So $U(\pi, D)$ is an unbiased estimate of $U(\pi, P)$.
- ▶ $\lambda(D) = \arg \max_{\pi} U(\pi, D)$ is a **learning algorithm**, picking the best classifier for a dataset D .
- ▶ Then $U(\lambda(D), D)$ is biased, as its expectation obeys:

$$\mathbb{E}_{D \sim P}[U(\lambda(D), D)] = \int_D dP(D) U(\lambda(D), D) \quad (1)$$

$$= \int_D dP(D) \max_{\pi} U(\pi, D) \quad (2)$$

$$\geq \int_D dP(D) U(\pi', D) \quad \forall \pi' \quad (3)$$

$$(4)$$

- ▶ i.e. the expected training accuracy is higher than that of any classifier.

Example of a biased and unbiased estimator: Training error

- ▶ $U(\pi, D)$ is the **measured** accuracy of a **classifier** π on **data** D
- ▶ $U(\pi, P) = \mathbb{E}_{D \sim P}[U(\pi, D)]$ is the **actual** accuracy on distribution P .
- ▶ So $U(\pi, D)$ is an unbiased estimate of $U(\pi, P)$.
- ▶ $\lambda(D) = \arg \max_{\pi} U(\pi, D)$ is a **learning algorithm**, picking the best classifier for a dataset D .
- ▶ Then $U(\lambda(D), D)$ is biased, as its expectation obeys:

$$\mathbb{E}_{D \sim P}[U(\lambda(D), D)] = \int_D dP(D) U(\lambda(D), D) \quad (1)$$

$$= \int_D dP(D) \max_{\pi} U(\pi, D) \quad (2)$$

$$\geq \int_D dP(D) U(\pi', D) \quad \forall \pi' \quad (3)$$

$$= U(\pi', P) \quad (4)$$

- ▶ i.e. the expected training accuracy is higher than that of any classifier.

Bias/Variance tradeoff

- ▶ Dataset $D \sim P$ (the dataset $D = \{(x_t, y_t) | t = 1, \dots, T\}$ is sampled from the unknown distribution P)
- ▶ Predictor $f_D(x)$ (it makes predictions about individual x)
- ▶ Target function $y = f(x) + \epsilon$ (we want $f_D \approx f$).
- ▶ $\mathbb{E}(\epsilon) = 0$ zero-mean noise with variance $\sigma^2 = \mathbb{V}(\epsilon)$

MSE decomposition

$$\mathbb{E}[(f - f_D)^2] = \mathbb{V}(f_D) + \mathbb{B}(f_D)^2 + \sigma^2$$

Variance

How sensitive the estimator is to the data

$$\mathbb{V}(f_D) = \mathbb{E}[(f_D - \mathbb{E}(f_D))^2]$$

Bias

What is the expected deviation from the true function

$$\mathbb{B}(f_D) \triangleq \mathbb{E}[(f_D - f)]$$

Generalisation in theory and practice

Learning and generalisation

Bias and variance

Bias/Variance tradeoff

Bias/Variance tradeoff

- Dataset $D \sim P$ (the dataset $D = \{(x_t, y_t) | t = 1, \dots, T\}$ is sampled from the unknown distribution P)
- Predictor $\hat{f}_D(x)$ (it makes predictions about individual x)
- Target function $y = f(x) + \epsilon$ (we want $\hat{f}_D \approx f$)
- $\mathbb{E}(\epsilon) = 0$ zero-mean noise with variance $\sigma^2 = \mathbb{V}(\epsilon)$

MSE decomposition

$$\mathbb{E}[(f - \hat{f}_D)^2] = \mathbb{V}(\hat{f}_D) + \mathbb{B}(\hat{f}_D)^2 + \sigma^2$$

Variance

How sensitive the estimator is to the data

$$\mathbb{V}(\hat{f}_D) = \mathbb{E}[(\hat{f}_D - \mathbb{E}(\hat{f}_D))^2]$$

Bias

What is the expected deviation from the true function

$$\mathbb{B}(\hat{f}_D) \triangleq \mathbb{E}[\hat{f}_D - f]$$

The bias/variance trade-off. This is the classical statistics formulation, applied to a regression estimator $\hat{f}_D$, not generally for $U(\pi, P)$. It's good background, and the proof below is worth doing for the practice, but it's only needed at the **MSc level**: the regret decomposition already covers what you need to know about generalisation error in the BSc course.

Example: mean estimation

- ▶ Data $D = x_1, \dots, x_T$ with $\mathbb{E}[x_t] = \mu$.
- ▶ Goal: estimate μ with some estimator f_D to minimise MSE: $\mathbb{E}[(x - f_D)^2]$, the expected square difference between new samples our guess.

Optimal estimate

To minimise the MSE, we use $f^* = \mu$. This gives us two ideas:

Empirical mean estimator:

- ▶ $f_D = \sum_{t=1}^T x_t / T$.
- ▶ $\mathbb{B}(f_D) = 0$. (unbiased estimator)
- ▶ $\mathbb{V}(f_D) = \sigma^2 / T$

Smoothed mean estimator:

- ▶ $f_D = (\lambda + \sum_{t=1}^T x_t) / (T + 1)$.
- ▶ $\mathbb{B}(f_D) = O(1/T)$.
- ▶ $\mathbb{V}(f_D) = \frac{T\sigma^2}{(T+1)^2}$

Generalisation in theory and practice

└ Learning and generalisation

└ Bias and variance

└ Example: mean estimation

Example: mean estimation

- Data $D = x_1, \dots, x_T$ with $\mathbb{E}[x_t] = \mu$.
- Goal: estimate μ with some estimator f_D to minimise MSE: $\mathbb{E}[(x - f_D)^2]$, the expected square difference between new samples our guess.

Optimal estimate

To minimise the MSE, we use $f^* = \mu$. This gives us two ideas:

Empirical mean estimator:

- $f_D = \sum_{t=1}^T x_t / T$.
- $\mathbb{B}(f_D) = 0$. (unbiased estimator)
- $\mathbb{V}(f_D) = \sigma^2 / T$

Smoothed mean estimator:

- $f_D = (\lambda + \sum_{t=1}^T x_t) / (T + 1)$.
- $\mathbb{B}(f_D) = O(1/T)$.
- $\mathbb{V}(f_D) = \frac{\sigma^2}{T(T+1)}$

Proofs. Due to the linearity of expectations, if $\mathbb{E}[x_t] = \mu$ for all t then:

$$\mathbb{E}[f_D] = \mathbb{E}\left[\sum_t x_t / T\right] = \sum_t \mathbb{E}[x_t] / T = \mu.$$

Consequently, $\mathbb{B}(f_D) = \mathbb{E}[f_D - \mu] = 0$. If x, y are independent then $\mathbb{V}(x + y) = \mathbb{V}(x) + \mathbb{V}(y)$. In addition, $\mathbb{V}(cx) = c^2 \mathbb{V}(x)$ for any constant c . Hence, if $\mathbb{V}[x_t] = \sigma^2$ for all t :

$$\mathbb{V}[f_D] = \mathbb{V}\left[\sum_t x_t / T\right] = \sum_t \mathbb{V}[x_t / T] = \sum_t \frac{\sigma^2}{T^2} = \frac{\sigma^2}{T}.$$

Smoothed estimator. We can rewrite the sum as $\frac{\lambda}{T+1} + \frac{T}{T+1} \times \sum_{t=1}^T x_t / T$. We can follow the previous proof, to show that there is a slight $O(T^{-1})$ bias. The variance is then also $\sigma^2 T / (T + 1)^2$, so very slightly lower.

A proof of the bias/variance trade-off

- ▶ RV's $y_t \sim P$, $\mathbb{E}[y_t] = \mu$, $y_t = \mu + \epsilon_t$.
- ▶ Estimator f_D , $D = y_1, \dots, y_{t-1}$.

$$\begin{aligned}\mathbb{E}[(f_D - y_t)^2] &= \mathbb{E}[f_D^2] - 2\mathbb{E}[f_D y_t] + \mathbb{E}[y_t^2] \\ &= \mathbb{V}[f_D] + \mathbb{E}[f_D]^2 - 2\mathbb{E}[f_D y_t] + \mathbb{E}[y_t^2] \\ &= \mathbb{V}[f_D] + \mathbb{E}[f_D]^2 - 2\mathbb{E}[f_D]\mathbb{E}[y_t] + \mathbb{E}[y_t^2] \\ &= \mathbb{V}[f_D] + \mathbb{E}[f_D]^2 - 2\mathbb{E}[f_D]\mu + \mathbb{E}[y_t^2] \\ &= \mathbb{V}[f_D] + \mathbb{E}[f_D]^2 - 2\mathbb{E}[f_D]\mu + \mathbb{E}[(\mu + \epsilon_t)^2] \\ &= \mathbb{V}[f_D] + \mathbb{E}[f_D]^2 - 2\mathbb{E}[f_D]\mu + \mathbb{E}[\mu^2 + 2\mu\epsilon_t + \epsilon_t^2] \\ &= \mathbb{V}[f_D] + \mathbb{E}[f_D]^2 - 2\mathbb{E}[f_D]\mu + \mu^2 + \sigma^2 \\ &= \mathbb{V}[f_D] + (\mathbb{E}[f_D] - \mu)^2 + \sigma^2 \\ &= \mathbb{V}(f_D) + \mathbb{B}(f_D)^2 + \sigma^2\end{aligned}$$

Generalisation in theory and practice

Learning and generalisation

Bias and variance

A proof of the bias/variance trade-off

A proof of the bias/variance trade-off

► RV's $y_i \sim P$, $\mathbb{E}[y_i] = \mu$, $y_i = \mu + \epsilon_i$.

► Estimator $\hat{f}_D$, $D = y_1, \dots, y_{D-1}$.

$$\begin{aligned}\mathbb{E}[(\hat{f}_D - y_i)^2] &= \mathbb{E}[\hat{f}_D^2] - 2\mathbb{E}[\hat{f}_D y_i] + \mathbb{E}[y_i^2] \\ &= \mathbb{V}[\hat{f}_D] + \mathbb{E}[\hat{f}_D]^2 - 2\mathbb{E}[\hat{f}_D] \mathbb{E}[y_i] + \mathbb{E}[y_i^2] \\ &= \mathbb{V}[\hat{f}_D] + \mathbb{E}[\hat{f}_D]^2 - 2\mathbb{E}[\hat{f}_D] \mu + \mathbb{E}[y_i^2] \\ &= \mathbb{V}[\hat{f}_D] + \mathbb{E}[\hat{f}_D]^2 - 2\mathbb{E}[\hat{f}_D] \mu + \mathbb{E}[(\mu + \epsilon_i)^2] \\ &= \mathbb{V}[\hat{f}_D] + \mathbb{E}[\hat{f}_D]^2 - 2\mathbb{E}[\hat{f}_D] \mu + \mathbb{E}[\mu^2 + 2\mu\epsilon_i + \epsilon_i^2] \\ &= \mathbb{V}[\hat{f}_D] + \mathbb{E}[\hat{f}_D]^2 - 2\mathbb{E}[\hat{f}_D] \mu + \mu^2 + \sigma^2 \\ &= \mathbb{V}[\hat{f}_D] + (\mathbb{E}[\hat{f}_D] - \mu)^2 + \sigma^2 \\ &= \mathbb{V}(\hat{f}_D) + \mathbb{B}(\hat{f}_D)^2 + \sigma^2\end{aligned}$$

Level notes

- **BSc**: Make sure you understand each and every step of the derivation.
- **MSc**: You should be able to re-use the ideas in this proof for similar proofs.

Generalisation error

Regret decomposition

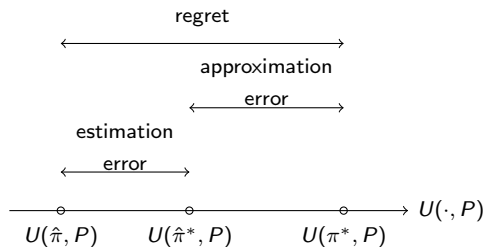
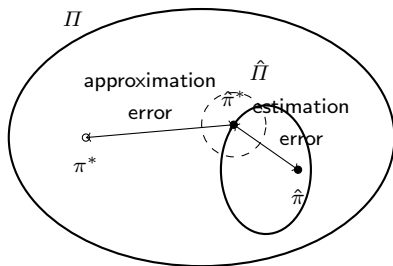
Let the optimal rule be $\pi^* \in \Pi$, the best approximate rule be $\hat{\pi}^* \in \hat{\Pi}$ and our rule be $\hat{\pi} \in \hat{\Pi}$. We call the difference between the performance of π^* and $\hat{\pi}$ our **regret**:

$$\underbrace{U(\pi^*, P) - U(\hat{\pi}, P)}_{\text{regret}} = \underbrace{U(\pi^*, P) - U(\hat{\pi}^*, P)}_{\text{approximation error}} + \underbrace{U(\hat{\pi}^*, P) - U(\hat{\pi}, P)}_{\text{estimation error}}$$

We can bound the regret by bounding each term separately.

- ▶ The **approximation error** tells us how expressive our class of rules is, i.e. how much we lose by looking at a restricted class $\hat{\Pi}$ of rules. It is similar to estimator **bias**.
- ▶ The **statistical error** tells us how well the empirical performance on D approximates the true performance. It is similar to estimator **variance**.
- ▶ As a rule of thumb, the larger our class $\hat{\Pi}$, the better the possible approximation but the higher the statistical error.

Geometric intuition



- ▶ The fact that to $\hat{\Pi} \neq \Pi$ creates the **approximation error**.
- ▶ π is a function of dataset D , which is random. This noise results in **estimation error** (dashed circle: where $\hat{\pi}$ could land, over different draws of D).
- ▶ Both panels show the **same** three rules. On the left, the gaps come from *where* the rules sit (inside or outside $\hat{\Pi}$, and where exactly within it). On the right, the same gaps are measured in the only unit that actually matters: utility.

Generalisation in theory and practice

Learning and generalisation

Generalisation

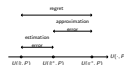
Geometric intuition

Geometric intuition



► The fact that to $\tilde{H} \neq H$ creates the approximation error.

► q is a function of dataset D , which is random. This noise results in estimation error (dashed circle: where $\tilde{H}$ could land, over different draws of D).



► Both panels show the same three rules. On the left, the gaps come from where the rules sit (inside or outside $\tilde{H}$, and where exactly within $\tilde{H}$). On the right, the same gaps are measured in the only unit that actually matters: utility.

Notes

- Understand which gap is "the model's fault" and which is "bad luck with the data".
- The approximation error only depends on the model class. We give a specific example below, where we can bound it no matter what the true model is.
- The estimation error is a random variable: a function of the randomly generated data D . While we do not know how much this can be, we can bound it with high probability.
- The discussion below is aimed at the **MSc** level.

Approximation error

- ▶ Our model limits us to a set of decision rules $\hat{\Pi} \neq \Pi$.
- ▶ The most we could do is find the best rule in $\hat{\Pi}$.
- ▶ This still leaves a gap:

$$\Delta \triangleq \max_{\pi \in \Pi} U(\pi, P) - \max_{\hat{\pi} \in \hat{\Pi}} U(\hat{\pi}, P)$$

The gap can be characterised in some cases.

Example: ϵ -net on Lipschitz $U(\cdot, P)$.

- ▶ Assume $U(\pi, P)$ is a Lipschitz function of π for all P , i.e. $|U(\pi, P) - U(\pi', P)| \leq Ld(\pi, \pi')$ for some metric d .
- ▶ Let $\hat{\Pi}$ be an ϵ -net on Π , i.e. $\max_{\pi \in \Pi} \min_{\pi' \in \hat{\Pi}} d(\pi, \pi') = \epsilon$.
- ▶ Then $\Delta \leq L\epsilon$.

Estimation error

- First, let us bound $U(\hat{\pi}^*, P) - U(\hat{\pi}, P)$ by making an assumption.
- Then, we can prove that our assumption holds with high probability.

Lemma

Let $f, g : S \rightarrow \mathbb{R}$. If $\|f - g\|_{\infty} \leq \epsilon$ and $f(x) \geq f(z)$, while $g(y) \geq g(z)$, for all z , i.e. x, y maximise f, g respectively

$$f(x) - f(y) \leq 2\epsilon.$$

This holds as: $f(x) - f(y) \leq g(x) + \epsilon - f(y) \leq g(y) + \epsilon - f(y) \leq 2\epsilon$.

Corollary

If $|U(\pi, P) - U(\pi, D)| \leq \epsilon$ for all π then

$$U(\hat{\pi}^*, P) - U(\hat{\pi}, P) \leq 2\epsilon$$

- Let us now prove that, with high probability, $|U(\pi, P) - U(\pi, D)| \leq \epsilon$.

Bounding the estimation error

For any fixed rule $\pi \in \Pi$ and utility function $U : \Pi \times \mathcal{X}^T \rightarrow [0, 1]$,

$$P^T(|U(\pi, D) - U(\pi, P)| \geq \epsilon) \leq 2 \exp(-2T\epsilon^2).$$

This is a direct application of Hoeffding's inequality¹. Taking the union bound over the set $\hat{\Pi}$ gives:

$$P^T(\exists \pi \in \hat{\Pi} : |U(\pi, D) - U(\pi, P)| \geq \epsilon) \leq 2|\hat{\Pi}| \exp(-2T\epsilon^2).$$

Setting the right side equal to δ and re-arranging,

$$P^T \left(\max_{\pi \in \hat{\Pi}} |U(\pi, D) - U(\pi, P)| \geq \sqrt{\frac{\ln(2|\hat{\Pi}|/\delta)}{2T}} \right) \leq \delta.$$

Example: ϵ -net.

In a n dimensional space we require $|\hat{\Pi}| = O(\epsilon^{-n})$. This means that our statistical error is $O(\sqrt{n \ln(1/\epsilon\delta)/T})$.

¹See Hoeffding's inequality in the confidence intervals presentation 

The finite hypothesis algorithm

- ▶ Input: a finite set of rules $\hat{\Pi}$, data D , utility U
- ▶ Return $\hat{\pi} \in \arg \max_{\pi \in \hat{\Pi}} U(\pi, D)$.

Regret of the finite hypothesis algorithm.

With probability $1 - \delta$

$$U(\hat{\pi}, P) \geq U(\hat{\pi}^*, P) - \sqrt{2 \ln(2|\hat{\Pi}|/\delta)/T} \quad (5)$$

$$U(\pi^*, P) - U(\hat{\pi}, P) \leq \Delta + \sqrt{2 \ln(2|\hat{\Pi}|/\delta)/T} \quad (6)$$

Examples

- ▶ ML estimation: $U(\beta, D) = P_\beta(D)$ is the data likelihood.
- ▶ Accuracy, etc: $U(\pi, D)$.

VC Dimension

Here we consider sets Π of deterministic rules $\pi : \mathcal{X} \rightarrow \{0, 1\}$.

Shattering

If a $S \subset \mathcal{X}$ can with $|S| = m$, can be assigned any labelling $y_1, \dots, y_m$ by a $\pi \in \Pi$, then we say Π shatters S .

The VC dimension

This is the largest-size set S that Π can shatter.

Example: Perceptrons on $\mathbb{R}^2$

This class has VC dimension 3 on the plane.

Quality metrics

Supervised machine learning problems

Metrics for model quality

Classification

Regression

Generalisation

Estimating quality

Methodology

Learning and generalisation

Introduction

Bias and variance

Generalisation

PAC Learning

The realisable setting

Binary classification

Learning algorithm λ

- ▶ Takes data $D = \{(x_t, y_t)\}$ as input
- ▶ Generates deterministic decision rules $\pi : X \rightarrow \{0, 1\}$,

The loss of a rule π .

- ▶ Assume an existing concept class $\pi^* \in \Pi$
- ▶ Distribution $x_t \sim P$ is i.i.d. and $x_1, \dots, x_T \sim P^T$.
- ▶ The loss under distribution P is

$$L(\pi) = P(\{x : \pi(x) \neq \pi^*(x)\})$$

Realisable PAC learner

- ▶ $\lambda : (\mathcal{X} \times \mathcal{Y})^* \rightarrow \Pi$ is (ϵ, δ) -PAC, if for any P and $\epsilon, \delta > 0$, and any concept $\pi^* \in \Pi$, there is T such that

$$P^T(\{D : L[\lambda(D)] > \epsilon\}) < \delta, \quad D = (\{x_t, \pi^*(x_t)\}), x_t \sim P.$$